

Claude-Ökosystem im Überblick: Code, Cowork, Features, Hacks

Quellen: [video-summary-sQBinJA_zxU.md](#) ("Claude Code vs. Claude Co-Work"),
[video-summary-B_OqkMRFonM.md](#) ("Jede Claude Funktion erklärt"),
[video-summary-wZeOwqmSw84.md](#) ("Learn 97% of Claude in Under 16 Minutes").

Praxistipps-Ergänzung: [video-summary-KZAJeq5n-m8.md](#), [video-summary-VKMNP_5vOmM.md](#),
[video-summary-XEbR5gmxGQ0.md](#), [video-summary-W30OHZdd7WQ.md](#),
[video-summary-LoMOPj-IO8U.md](#), [video-summary-IYzgxWs4sZ4.md](#),
[video-summary-JDebq_fxLlw.md](#), [video-summary-zB9it8-nbeM.md](#)

Drei Videos decken sich stark in ihrem Ziel — einen vollständigen Überblick über Claudes Funktionsumfang zu geben — nutzen aber unterschiedliche Gliederungen. Hier zusammengeführt statt dreifach redundant.

Die zwei Philosophien: Claude Code vs. Claude Cowork

Aus sQBinJA_zxU, mit der treffendsten Kernaussage: "**Code ist der Maschinenraum, Cowork die Chefetage.**"

- **Claude Code** — Terminal-basiert, voller Systemzugriff, für tiefe technische Eingriffe (Datenbanken aufsetzen, Git, Netzwerk). Sehr token-effizient, da keine grafische Oberfläche geladen wird — relevant, weil ein Agent-Loop teils 50-100 Iterationen brauchen kann.
- **Claude Cowork** — nicht zum Software-Bauen gedacht, sondern zur Orchestrierung von Alltags-Workflows (Mail, Kalender, Dateisortierung). Hat persistentes Gedächtnis über Sessions hinweg.

Der zugrundeliegende Mechanismus in beiden ist der **Agent Loop**: Kontextualisierung → Planung → Ausführung → Validierung.

Die Bauteile, geordnet (Roboter-Metapher aus B_OqkMRFonM)

Arbeitsplatz — Workspace-Ordner, Claude.md (projektspezifisch), globale Anweisungen (app-weit), Projects (Ordner mit eigenem Memory/Kontext)

Gehirn — Memory (Langzeitgedächtnis, automatisch + manuell), Kontextfenster (Kurzzeitgedächtnis, für neue große Aufgaben lieber neuen Chat starten), Extended Thinking (mehr Nachdenkzeit gegen mehr Nutzungskontingent)

Augen/Sensorik — Multimodal (Screenshots, PDFs, Tabellen; kein Video laut Video), Websuche, Recherche (Deep-Research-artige Reports mit Quellen), Connectors (Gmail, Calendar, Notion, Slack), MCP (die offene Schnittstelle dahinter — "der USB-Anschluss für KI-Tools", ausführlich in [mcp-ueberblick.md](#))

Hände — Artifacts (interaktive Mini-Apps im Chat), Chrome-Erweiterung (Browser-Steuerung), Computer Use (volle Rechnersteuerung, laut beiden Videos noch nicht sehr schnell)

Autopilot — Skills (wiederverwendbare Arbeitsabläufe, laut B_OqkMRFonM "der wichtigste Punkt"), Slash-Commands (Skill-Trigger), Plugins (Skill-Bündel), Scheduled Tasks/Automations, Dispatch Mode (Mobile → Desktop-Steuerung)

Ergänzende Hacks aus wZeOwqmSw84 (nicht in den anderen beiden enthalten)

- **Memory Import** — ChatGPT-Verlauf/Präferenzen exportieren und in Claude importieren
- **Model Selector** — Haiku (schnell/günstig) vs. Sonnet (Alltag) vs. Opus (komplexe Analysen), alternativ "Adaptive Thinking"
- **Voice Mode** — Diktieren statt Tippen
- **Claude Channels** — Anbindung an iMessage/Telegram/Discord (nicht gegengecheckt)
- **Claude Design** (claude.ai/design) — Pitch-Decks, Landingpages, Mockups, Motion Graphics, mit Adobe-Partnerschaft (nicht gegengecheckt)

Praktische Einordnung

Skills sind laut sQBinJA_zxU technisch mehr als Custom Instructions: echte lokale Ordner mit Markdown + Skripten. Das Kontextfenster-Problem wird über **Progressive Disclosure** gelöst — nur die per semantischer Suche passende Skill-Datei wird geladen, nicht das ganze "Firmenhandbuch" auf einmal. Ausführliches Grundkonzept inkl. Beispielen und Bauanleitung:

[claude-skills-ueberblick.md](https://github.com/sQBinJA/claude-skills-ueberblick.md).

Praxistipps aus weiteren Videos

Neun zusätzliche Videos liefern konkrete, im Kern verifizierbare Praxistipps zum Alltag mit Claude Code — ergänzend zum Grundkonzept oben:

- **Kostenoptimierung** ([video-summary-KZAJeq5n-m8.md](#), [video-summary-XEbR5qmxGQ0.md](#)): Kontextfenster-Sichtbarkeit (Status Line, /context), Prompt-Caching-Mechanik (Rabatt verfällt bei Modellwechsel, Reasoning-Level-Wechsel oder Inaktivität), Subagent-Orchestrierung, Handover-Dateien statt automatischer Compaction statt Mikro-Optimierungen
- **Output-Styles** ([video-summary-VKMNP 5vOmM.md](#)): Gegen Opus 5s Hang zu verschachtelten, jargonlastigen Antworten helfen selbst definierte Styles (.claude/settings.local.json), testbar per /rewind und Branch
- **Effort-Stufe als Kostenhebel** ([video-summary-zB9it8-nbeM.md](#)): Fable 5.1 führt auf höchsten Effort-Einstellungen (X-High/Max) bei sehr hohem Token-Verbrauch, ist auf mittleren Einstellungen aber kostenmäßig konkurrenzfähig zu GPT-5.6 Sol/Grok 4.6 — die Effort-Wahl entscheidet über die Kosten, nicht das Modell an sich

- **Reasoning-Effort bei Coding** ([video-summary-IYzgXWs4sZ4.md](https://www.youtube.com/watch?v=IYzgXWs4sZ4)): Opus 5 performt bei Coding-Benchmarks teils *besser* mit Medium- statt Extra-High-Reasoning-Effort (Over-Engineering-Risiko bei zu viel Denkzeit)
- **Anthropic Academy** ([video-summary-W30OHZdd7WQ.md](https://www.youtube.com/watch?v=W30OHZdd7WQ)): Kostenlose Kurse mit Zertifikat, Pflicht-Empfehlung für "AI Fluency: Frameworks and Foundations" (Kapitel Diligence), "Claude Code in Action" und "Introduction to Subagents"; API-Kurs und Cloud-Plattform-Kurse (Bedrock/Vertex) für die meisten optional
- **INTENT.md / AI-Native SDLC** ([video-summary-LoMOPj-IO8U.md](https://www.youtube.com/watch?v=LoMOPj-IO8U)): Anthropic's Blogpost formalisiert den Entwicklungszyklus als Kette versionierter Artefakte (intent.md → spec.md → plan.md → PR-Review → Maintenance-Loop) — ausdrücklich eine von mehreren validen Herangehensweisen, kein zwingender Ersatz für bestehende Workflows
- **MCP-Tool-Integration** ([video-summary-JDebq_fxLlw.md](https://www.youtube.com/watch?v=JDebq_fxLlw)): Higgsfield bündelt 30+ KI-Bild-/Videomodelle per MCP-Server direkt in Claude Code — Nutzen hängt am Credit-Verbrauch, "Unlimited"-Pläne greifen im Terminal-Workflow nicht

Prompting Claude 5: 7 Regeln (hEWqRbQe6z4)

Zusammenfassung des Opus-5-Prompting-Guides (Guide und Umgebungsvariablen per WebSearch bestätigt): ganze Aufgabe vorab nennen; Interview zuerst (Community-Technik, keine Anthropic-Regel); das "Warum" nennen; Definition of Done angeben; begründete statt starrer Regeln; alte Tricks (u. a. Selbstverifikation im Prompt) weglassen; Antwortstil zentral in der CLAUDE.md. Überschneidet sich mit [ai-agent-workflow.md](https://www.youtube.com/watch?v=ai-agent-workflow) (Interview-Prompt) und [context-rot-ueberblick.md](https://www.youtube.com/watch?v=context-rot-ueberblick) (Systemprompt-Kürzung). Mögliche Spannung: Regel 6 gegen "immer gegenprüfen" — aufgelöst als "Selbstverifikation im Prompt ist redundant, menschliche Prüfung bleibt". Im Video genanntes "Haiku 5.1" ließ sich nicht bestätigen.

Nachtrag Faktencheck (2026-09-29): Die "7 Regeln" sind die Zählung des Hosts; im Guide belegt sind Verifikations-Anweisungen entfernen, Scope, Verbosity-Beispiel und Subagent-Limits (CLAUDE_CODE_MAX_SUBAGENT_SPAWN_DEPTH Default 3, CLAUDE_CODE_MAX_CONCURRENT_SUBAGENTS Default 20). "Haiku 5.1" existiert vermutlich nicht (Fable 5.1 und Mythos 5.1 erschienen am 01.09.). Siehe [video-summary-hEWqRbQe6z4.md](https://www.youtube.com/watch?v=hEWqRbQe6z4).

Offene Fragen (nicht gegengecheckt)

- Genaue aktuelle Modellnamen/-versionen, wie in den Videos gezeigt (Screenshots sind Momentaufnahmen)
- Existenz/Funktionsweise von "Claude Dispatch", "Claude Channels" und "Claude Design" als eigenständige, benannte Produktfeatures — Videos könnten informelle Namen für tatsächliche Anthropic-Features verwenden
- Wissensstand-Cutoff "Mai 2025", den B_OqkMRFonM nennt